

Pure Logic Coding Interview Master

Candidate: Irshath Ahamed | Senior Flutter Developer (Portal/Mobile)

Pure Logic Coding Interview Master — Analytical & Algorithmic Challenges

This handbook focuses strictly on **pure logical and analytical challenges**—problems that test your mathematical intuition, edge-case reasoning, state-machine modeling, and greedy/dynamic thinking in pure Dart without relying on external packages.

Index of Pure Logic Problems

1. [Logic 1: Trapping Rain Water \(Two-Pointer Elevation Logic\)](#).
2. [Logic 2: Product of Array Except Self \(Without Division Operator\)](#).
3. [Logic 3: Gas Station Circular Tour \(Greedy Surplus/Deficit Logic\)](#).
4. [Logic 4: Jump Game \(Greedy Reachability Range\)](#).
5. [Logic 5: Best Time to Buy and Sell Stock \(State Machine & Peaks/Valleys\)](#).
6. [Logic 6: First Missing Positive Integer \(\$O\(N\)\$ Time, \$O\(1\)\$ Space Cycle Sort\)](#).
7. [Logic 7: Container With Most Water \(Two-Pointer Inward Shrink Logic\)](#).
8. [Logic 8: Spiral Matrix 2D Traversal \(Boundary Peeling Logic\)](#).
9. [Logic 9: Roman Numeral to Integer & Integer to Roman \(Subtractive Rule\)](#).
10. [Logic 10: Count Number of Connected Worksite Islands \(Matrix Flood-Fill Logic\)](#).

Logic 1: Trapping Rain Water

The Problem:

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

- **Example:** `height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` $\rightarrow$ Output: 6

The Core Logic:

Water trapped above any bar i depends strictly on: $\text{Water at } i = \min(\text{max height to left}, \text{max height to right}) - \text{height}[i]$ Instead of computing left and right arrays with $O(N)$ auxiliary memory, we use **Two Pointers** from both ends (`left` , `right`). Whichever side has the smaller maximum height is the bottleneck and can be computed immediately.

Full Dart Implementation:

```
import 'dart:math';

int trapRainWater(List<int> height) {
  if (height.isEmpty) return 0;

  int left = 0;
  int right = height.length - 1;
  int leftMax = 0;
  int rightMax = 0;
  int totalWater = 0;

  while (left < right) {
    if (height[left] < height[right]) {
      // Left side is the bottleneck
      if (height[left] >= leftMax) {
        leftMax = height[left];
      } else {
        totalWater += leftMax - height[left];
      }
      left++;
    } else {
      // Right side is the bottleneck
      if (height[right] >= rightMax) {
        rightMax = height[right];
      } else {
        totalWater += rightMax - height[right];
      }
      right--;
    }
  }

  return totalWater;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$

Logic 2: Product of Array Except Self

The Problem:

Given an array `nums`, return an array `output` such that `output[i]` is equal to the product of all elements of `nums` except `nums[i]`. **Strict Rule:** You must write an algorithm that runs in $O(N)$ time and **without using the division operation** `/`.

- **Example:** `[1, 2, 3, 4]` $\rightarrow$ `[24, 12, 8, 6]`

The Core Logic:

Every result is simply the product of **all numbers to its left** multiplied by **all numbers to its right**:

$$\text{output}[i] = \text{prefixProduct}[i-1] \times \text{suffixProduct}[i+1]$$
 We pass left-to-right to populate prefix products directly into the result array, then pass right-to-left accumulating the suffix product in a single scalar variable.

Full Dart Implementation:

```
List<int> productExceptSelf(List<int> nums) {
    final n = nums.length;
    final List<int> result = List<int>.filled(n, 1);

    // 1. Pass Left-to-Right: Compute Prefix Products
    int prefix = 1;
    for (int i = 0; i < n; i++) {
        result[i] = prefix;
        prefix *= nums[i];
    }

    // 2. Pass Right-to-Left: Multiply by Suffix Products
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        result[i] *= suffix;
        suffix *= nums[i];
    }

    return result;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$ extra memory (excluding output array)

Logic 3: Gas Station Circular Tour

The Problem:

There are n gas stations along a circular route. You have a car with an unlimited gas tank, and it costs $\text{cost}[i]$ of gas to travel from station i to $i + 1$. You begin the journey with an empty tank at one of the gas stations. Return the starting gas station index if you can travel around the circuit once clockwise, otherwise return -1 .

- **Example:** $\text{gas} = [1, 2, 3, 4, 5]$, $\text{cost} = [3, 4, 5, 1, 2]$ $\rightarrow$ Output: 3 (Station 3: 4 gas, cost 1)

The Core Logic:

1. If $\sum \text{gas} < \sum \text{cost}$, it is mathematically impossible to complete the circuit regardless of where you start $\rightarrow$ return -1 .
2. If total gas $\geq$ total cost, **a valid starting station is guaranteed to exist.**
3. If tank drops below 0 while travelling from station A to station B , no station between A and B can possibly be the starting point! We greedily reset the starting station to $B + 1$ and reset our current tank to 0.

Full Dart Implementation:

```
int canCompleteCircuit(List<int> gas, List<int> cost) {
    int totalTank = 0;
    int currentTank = 0;
    int startingStation = 0;

    for (int i = 0; i < gas.length; i++) {
        final netGain = gas[i] - cost[i];
        totalTank += netGain;
        currentTank += netGain;

        // If current tank drops below zero, station i cannot reach i+1
        if (currentTank < 0) {
            // Pick next station as candidate and reset current tank
            startingStation = i + 1;
            currentTank = 0;
        }
    }

    // If total gas collected is less than total cost, impossible
    return totalTank >= 0 ? startingStation : -1;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$

Logic 4: Jump Game

The Problem:

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your **maximum jump length** at that position. Return `true` if you can reach the last index, or `false` otherwise.

- **Example 1:** `[2, 3, 1, 1, 4]` $\rightarrow$ `true`
- **Example 2:** `[3, 2, 1, 0, 4]` $\rightarrow$ `false` (Stuck at index 3 with jump 0)

The Core Logic:

Use a **Greedy Reachability Window**. Track the maximum index you can possibly reach (`maxReachable`). At each step `i`:

- If `i > maxReachable`, you are stranded $\rightarrow$ return `false`.
- Otherwise update `maxReachable = max(maxReachable, i + nums[i])`.
- If `maxReachable >= lastIndex`, return `true`.

Full Dart Implementation:

```
import 'dart:math';

bool canJump(List<int> nums) {
  int maxReachable = 0;
  final lastIndex = nums.length - 1;

  for (int i = 0; i < nums.length; i++) {
    // If current index is beyond the furthest reached point, cannot proceed
    if (i > maxReachable) return false;

    maxReachable = max(maxReachable, i + nums[i]);

    if (maxReachable >= lastIndex) return true;
  }

  return true;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$

Logic 5: Best Time to Buy and Sell Stock

Problem (Part 1 - Single Transaction):

Maximize profit by choosing a single day to buy and a single future day to sell.

- **Example:** `[7, 1, 5, 3, 6, 4]` $\rightarrow$ Buy on day 2 (\$1), sell on day 5 (\$6), profit = \$5.

Problem (Part 2 - Multiple Transactions):

You may complete as many buy/sell transactions as you like (e.g., buy one, sell one multiple times).

The Core Logic:

- **Part 1:** Track the `minPrice` seen so far. At each day, calculate profit if sold today (`price - minPrice`), and keep the maximum.
- **Part 2:** Sum up every positive price difference between adjacent days: whenever `prices[i] > prices[i-1]` , capture the gain (`prices[i] - prices[i-1]`).

Full Dart Implementation:

```
import 'dart:math';

// Part 1: Single Transaction Max Profit
int maxProfitSingle(List<int> prices) {
  int minPrice = double.maxFinite.toInt();
  int maxProfit = 0;

  for (final price in prices) {
    if (price < minPrice) {
      minPrice = price;
    } else {
      maxProfit = max(maxProfit, price - minPrice);
    }
  }

  return maxProfit;
}

// Part 2: Multiple Transactions (Greedy Accumulation)
int maxProfitMultiple(List<int> prices) {
  int totalProfit = 0;

  for (int i = 1; i < prices.length; i++) {
    if (prices[i] > prices[i - 1]) {
      totalProfit += prices[i] - prices[i - 1];
    }
  }

  return totalProfit;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$

Logic 6: First Missing Positive Integer

The Problem:

Given an unsorted integer array `nums`, return the smallest positive integer that is not present in the array. **Strict Constraint:** You must implement an algorithm that runs in $O(N)$ time and uses $O(1)$ auxiliary space. (You cannot sort because sorting takes $O(N \log N)$; you cannot use a HashSet because that takes $O(N)$ space).

- **Example 1:** `[1, 2, 0]` $\rightarrow$ `3`
- **Example 2:** `[3, 4, -1, 1]` $\rightarrow$ `2`
- **Example 3:** `[7, 8, 9, 11, 12]` $\rightarrow$ `1`

The Core Logic (Cycle Sort / Index as Hash Key):

For an array of length N , the answer MUST be between `1` and `N + 1`. We place each number `x` into its correct zero-indexed position `x - 1` (e.g. `1` goes to index `0`, `2` goes to index `1`, `3` goes to index `2`). Then we scan the array: the first index `i` where `nums[i] != i + 1` reveals the missing integer: `i + 1`.

Full Dart Implementation:

```
int firstMissingPositive(List<int> nums) {
    final n = nums.length;

    for (int i = 0; i < n; i++) {
        // While nums[i] is in valid range [1, n] and not already at its correct index:
        while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
            // Swap nums[i] with the element at its target index (nums[i] - 1)
            final targetIndex = nums[i] - 1;
            final temp = nums[i];
            nums[i] = nums[targetIndex];
            nums[targetIndex] = temp;
        }
    }

    // Find the first index where value != index + 1
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            return i + 1;
        }
    }

    // If all numbers 1..n are present, answer is n + 1
    return n + 1;
}
```

- **Complexity:** Time: $O(N)$ (each element is swapped at most twice) | Space: $O(1)$ in-place

Logic 7: Container With Most Water

The Problem:

Given an integer array `height` of length `n`, find two lines that together with the x-axis form a container that contains the most water.

- **Formula:** $\text{Area} = \min(\text{height}[i], \text{height}[j]) \times (j - i)$

The Core Logic:

Start with the widest possible container: pointers at `left = 0` and `right = n - 1`. The area is limited by the **shorter** bar. Moving the taller bar inward can never increase the area (width decreases, height is still bounded by the shorter bar). Therefore, we **always move the pointer pointing to the shorter bar inward**.

Full Dart Implementation:

```
import 'dart:math';

int maxArea(List<int> height) {
  int left = 0;
  int right = height.length - 1;
  int maxWater = 0;

  while (left < right) {
    final width = right - left;
    final currentHeight = min(height[left], height[right]);
    maxWater = max(maxWater, width * currentHeight);

    // Greedily discard the smaller bar
    if (height[left] < height[right]) {
      left++;
    } else {
      right--;
    }
  }

  return maxWater;
}
```

- **Complexity:** Time: $O(N)$ | Space: $O(1)$

Logic 8: Spiral Matrix 2D Traversal

The Problem:

Given an $m \times n$ matrix, return all elements of the matrix in **spiral order** (clockwise).

- **Example:**

```
[
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]
```

$\rightarrow$ Output: `[1, 2, 3, 6, 9, 8, 7, 4, 5]`

The Core Logic (Boundary Peeling):

Define 4 boundaries: `top`, `bottom`, `left`, `right`. Traverse 4 directions sequentially:

1. `left` $\rightarrow$ `right` along `top`, then increment `top`.
2. `top` $\rightarrow$ `bottom` along `right`, then decrement `right`.
3. `right` $\rightarrow$ `left` along `bottom`, then decrement `bottom`.
4. `bottom` $\rightarrow$ `top` along `left`, then increment `left`. Repeat while `top <= bottom` and `left <= right`.

Full Dart Implementation:

```
List<int> spiralOrder(List<List<int>> matrix) {
    if (matrix.isEmpty) return [];

    final List<int> result = [];
    int top = 0;
    int bottom = matrix.length - 1;
    int left = 0;
    int right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        // 1. Traverse Right
        for (int j = left; j <= right; j++) {
            result.add(matrix[top][j]);
        }
        top++;

        // 2. Traverse Down
        for (int i = top; i <= bottom; i++) {
            result.add(matrix[i][right]);
        }
        right--;

        // 3. Traverse Left (Check top <= bottom boundary)
        if (top <= bottom) {
            for (int j = right; j >= left; j--) {
                result.add(matrix[bottom][j]);
            }
            bottom--;
        }

        // 4. Traverse Up (Check left <= right boundary)
        if (left <= right) {
            for (int i = bottom; i >= top; i--) {
                result.add(matrix[i][left]);
            }
            left++;
        }
    }

    return result;
}
```

- **Complexity:** Time: $O(M \times N)$ | Space: $O(1)$ extra memory

Logic 9: Roman Numeral Converter

The Problem:

Convert a Roman numeral string to an integer, and convert an integer to a Roman numeral.

- **Symbols:** I=1, V=5, X=10, L=50, C=100, D=500, M=1000
- **Subtractive Rule:** If a smaller numeral precedes a larger numeral (e.g. IV=4, IX=9, XL=40, XC=90, CD=400, CM=900), subtract its value instead of adding it.

Full Dart Implementation:

```
int romanToInt(String s) {
  final Map<String, int> values = {
    'I': 1, 'V': 5, 'X': 10, 'L': 50,
    'C': 100, 'D': 500, 'M': 1000,
  };

  int total = 0;
  for (int i = 0; i < s.length; i++) {
    final current = values[s[i]] ?? 0;
    final next = (i + 1 < s.length) ? (values[s[i + 1]] ?? 0) : 0;

    if (current < next) {
      // Subtractive rule applies
      total -= current;
    } else {
      total += current;
    }
  }

  return total;
}

String intToRoman(int num) {
  final List<int> values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1];
  final List<String> symbols = ['M', 'CM', 'D', 'CD', 'C', 'XC', 'L', 'XL', 'X', 'IX', 'V', 'IV', 'I'];

  final buffer = StringBuffer();
  for (int i = 0; i < values.length && num > 0; i++) {
    while (num >= values[i]) {
      buffer.write(symbols[i]);
      num -= values[i];
    }
  }

  return buffer.toString();
}
```

- **Complexity:** Time: $O(1)$ (Bounded by max Roman numeral value 3999) | Space: $O(1)$

Logic 10: Count Connected Worksite Islands (Matrix Flood-Fill)

The Problem:

Given an $m \times n$ 2D binary grid where '1' represents land (active worksite zone) and '0' represents water/void, return the number of distinct islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

The Core Logic:

Iterate through every cell (r, c) . When a '1' is encountered:

1. Increment `islandCount`.

2. Trigger a **Depth-First Search (DFS) / Flood-Fill** to sink the entire connected island (mutating visited '1' s into '0' s so they are never counted again).

Full Dart Implementation:

```
int numIslands(List<List<String>> grid) {
    if (grid.isEmpty) return 0;

    int islandCount = 0;
    final rows = grid.length;
    final cols = grid[0].length;

    void dfs(int r, int c) {
        // Boundary and water checks
        if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] == '0') {
            return;
        }

        // Mark current cell as visited by sinking it
        grid[r][c] = '0';

        // Flood-fill in 4 directions
        dfs(r + 1, c); // Down
        dfs(r - 1, c); // Up
        dfs(r, c + 1); // Right
        dfs(r, c - 1); // Left
    }

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                islandCount++;
                dfs(r, c); // Sink the whole island
            }
        }
    }

    return islandCount;
}
```

- **Complexity:** Time: $O(M \times N)$ | Space: $O(M \times N)$ recursion stack depth